SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

문제풀이 1

Python

학습시작

당신은 스마트 시스템을 설계하는 개발자라고 가정해봅시다.

여러 도시에서 수집된 데이터를 분석해
도시 교통 흐름, 전력망 설계, 보안 시스템, 건물 구조물의 취약점 분석 등
복잡한 문제를 빠르고 정확하게 해결해야 합니다.

오늘 풀 4개의 문제는, 각기 다른 환경에서 정보를
‘탐색하고 분석’하거나 ‘변화를 시뮬레이션’하면서
의사결정을 내리는 데 꼭 필요한 역량을 키워줍니다.

학습시작

당신은 스마트 시스템을 설계하는 개발자라고 가정해봅시다.

1. 사탕 수열 문제 (20551) : 데이터 흐름 중 '일정한 증가 패턴'을 [ ] 하는 기술→ 이상 징후 탐지, 재고 분석, 온도 변화 감지에 활용
2. 전봇대 문제 (10580) : [ ] 을 통해 선이 겹치지 않게 배선하는 방법→ 회로 설계, 통신망 구성, 배선 충돌 방지에 활용
3. 탈주범 문제 (1953) : [ ] 를 활용하여 제한된 구조에서 이동 가능한 경로 탐색→ 로봇 경로 계획, 미로 탐색, 보안 영역 침입 탐지에 활용
4. 벽돌깨기 문제 (5656) : BFS/DFS를 활용하여 하나의 행동이 주변에 미치는 영향을 [ ] → 재난 예측, 구조물 붕괴 예측, 게임 물리 엔진 설계에 활용

- 이 문제들을 통해 데이터를 분석하고, 규칙을 찾고, 변화를 예측하는 컴퓨팅 사고력을 기르게 됩니다.

Objectives

학습목표

- 수열과 배열에서 조건을 찾아내는 탐색 능력을 기른다.
- 완전 탐색과 시뮬레이션의 흐름을 이해한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

문제풀이 1

- 20551. 증가하는 사탕 수열
- 10580. 전봇대
- 1953. 탈주범 검거
- 5656. 벽돌 깨기

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

문제풀이 1

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

20551.
증가하는 사탕 수열

문제 설명

세현이는 세 개의 상자를 나란히 세워 놓았다.

첫 번째 상자에는 사탕 A개, 두 번째 상자에는 사탕 B개, 세 번째 상자에는 사탕 C개가 들어 있다.

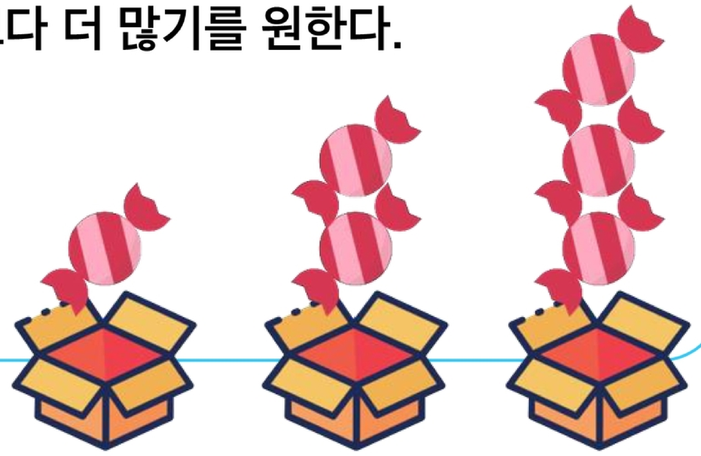
세현이는 **각 상자에 들어 있는 사탕의 개수가 순증가하기를 원한다.**

즉, 두 번째 상자에 들어 있는 사탕의 개수가 첫 번째 상자에 들어 있는 사탕의 개수보다 더 많기를 원하고,

세 번째 상자에 들어 있는 사탕의 개수가 두 번째 상자에 들어 있는 사탕의 개수보다 더 많기를 원한다.

또한, 세현이는 모든 상자가 비어 있지 않기를 원한다.

즉, **모든 상자에 1개 이상의 사탕이 들어 있기를 원한다.**

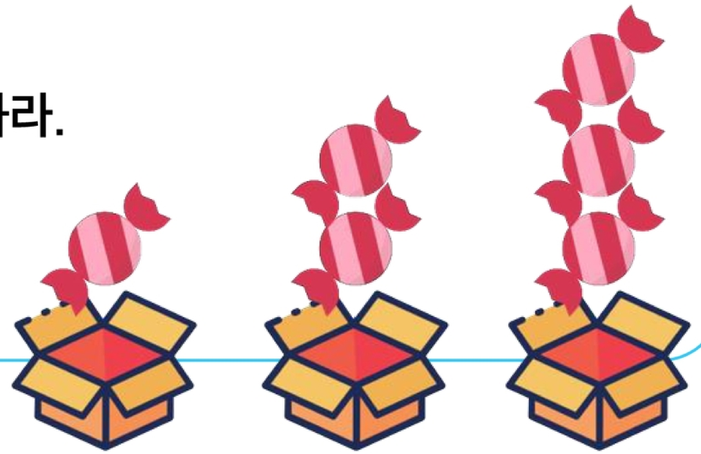


〈그림1_sw문제해결 응용_문제풀이1_증가하는 사탕 수열〉

문제 설명

세현이가 이 두 조건을 만족시키기 위해 할 수 있는 일은,
상자 속에 들어 있는 사탕을 (0개 이상) 먹어 없애 버리는 것이다.

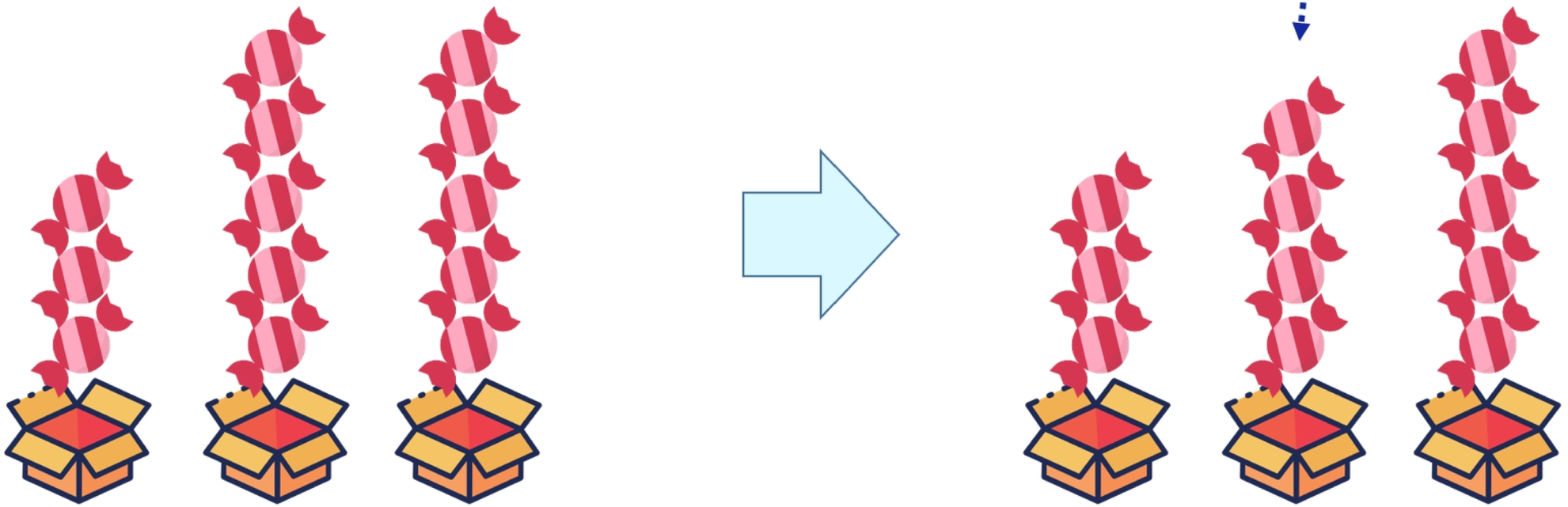
세현이가 조건을 만족시킬 수 있는지 판단하고,
만족시킬 수 있다면 **최소 몇 개의 사탕을 먹어야 하는지** 구하는 프로그램을 작성하라.



예제 1

- 입력: 3 5 5

2번 박스의 사탕을 1개 먹으면 된다.

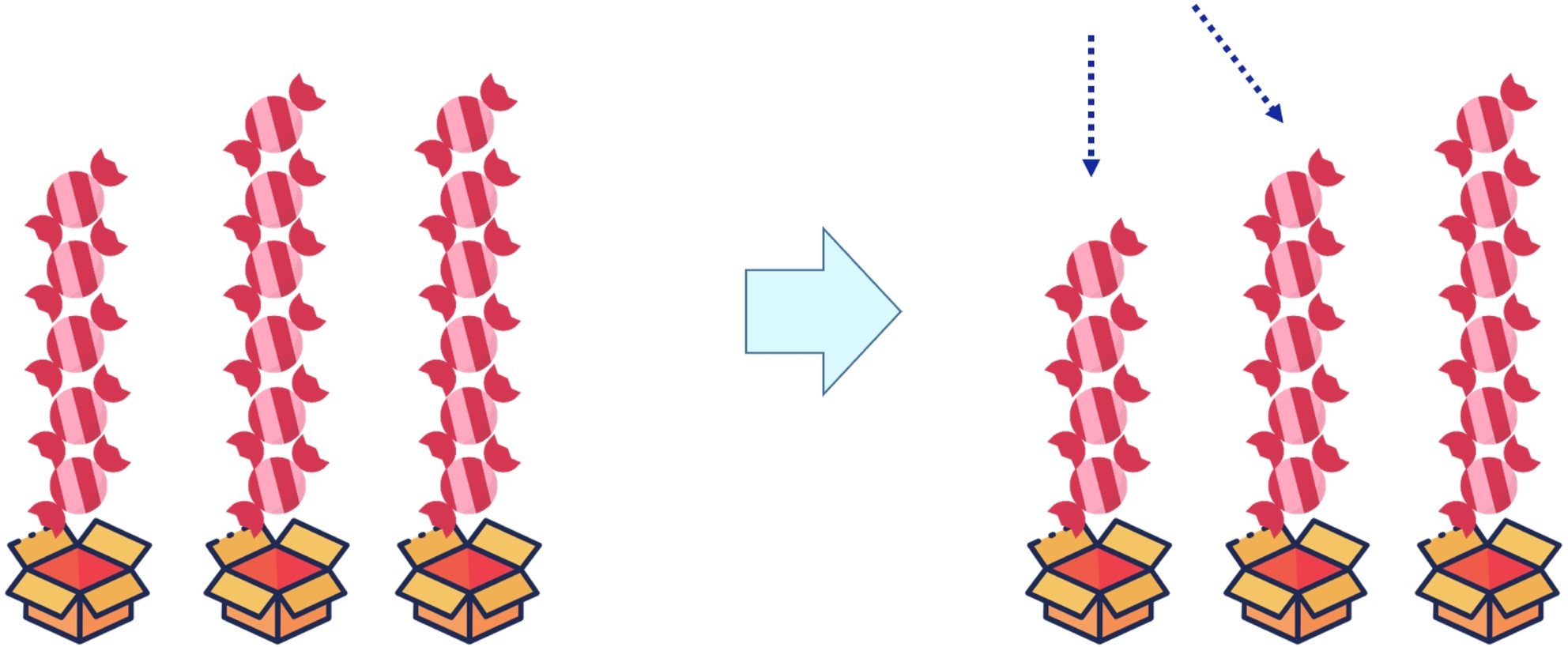


<그림2_sw문제해결 응용_문제풀이1_증가하는 사탕 수열 예제 1>

예제 2

- 입력: 5 6 6

2번 박스의 사탕을 1개 먹으면 2, 3번 박스는 조건을 만족한다.
하지만, 1, 2번 박스가 조건이 만족하지 않기 때문에
1번 박스 사탕도 1개를 먹어야 정답이 나온다.



<그림3_sw문제해결 응용_문제풀이1_증가하는 사탕 수열 예제 2>

이론

입력

- 첫 번째 줄에 테스트 케이스의 수 T가 주어진다.
- 각 테스트 케이스는 한 개의 줄로 이루어진다.
- 각 줄에는 세 개의 자연수 A,B,C (1≤A,B,C≤3000)가 공백 하나를 사이로 두고 주어진다.

출력

- 각 테스트 케이스마다,
  - ✓ 세현이가 사탕을 먹어치워서 조건을 만족시킬 수 없다면 -1을 출력한다.
  - ✓ 만족시킬 수 있다면, 최소 몇 개를 먹어야 하는지 출력한다.

입력과 출력 예시

입력

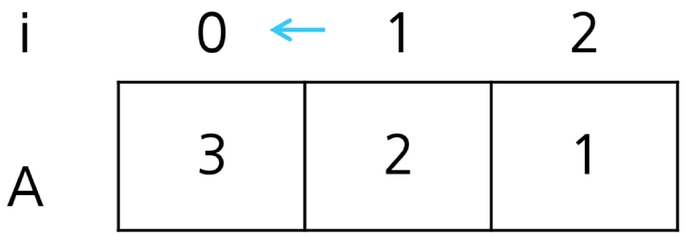
```
4
3 2 1
1 2 3
3 5 5
5 6 6
```

출력

```
#1 -1
#2 0
#3 1
#4 2
```

접근 방법

- A[i] < A[i+1]을 만족하도록 A[i] 사탕을 먹고, 그 개수를 센다.
- 단, A[i]가 0이 되면 조건을 만족할 수 없다.
- 반복문으로 구현하면 상자가 더 많은 경우도 같은 방법으로 해결할 수 있다.



10580. 전봇대

문제 설명

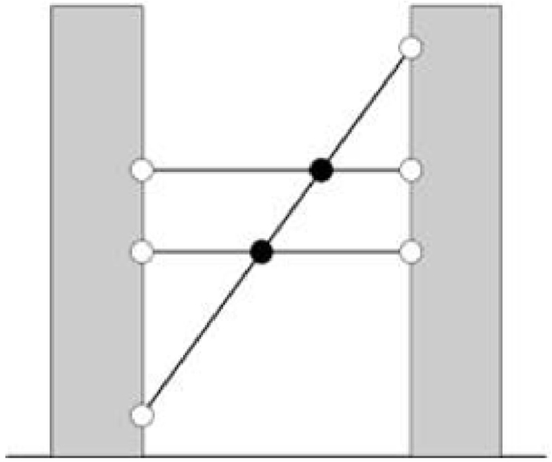
현우는 길을 가다가 전선들이 복잡하게 꼬여 있는 전봇대 두 개를 보았다.

두 전봇대는 높이가 매우 높으며, N개의 팽팽한 전선으로 연결되어 있었다.

두 전선이 끝점이 같은 경우는 없으나, 교차하는 경우는 있다.

이를 그림으로 하면 오른쪽과 같다.

(전선 3개가 있으며, 교차점 2개가 검은색으로 칠해졌다.)



<그림4_sw문제해결 응용_문제풀이1_전봇대>

세 개 이상의 전선이 하나의 점에서 만나지 않는다고 가정하자. 이 전봇대에는 총 몇 개의 교차점이 있을까?

이론

입력

- 첫 번째 줄에 테스트 케이스의 수 TC가 주어진다.
- 이후 TC개의 테스트 케이스가 새 줄로 구분되어 주어진다.
- 각 테스트 케이스는 다음과 같이 구성되었다.
- 첫 번째 줄에 주어지는 전선의 개수 N이 주어진다 ($1 \le N \le 1000$).
- 이후 N개의 줄에 두 양의 정수 Ai, Bi 가 주어진다. ($1 \le Ai, Bi \le 10000$)
- 이는 i번째 전선이, 첫 번째 전봇대의 Ai cm 고도에 걸려 있고, 두 번째 전봇대의 Bi cm 고도에 걸려 있음을 뜻한다.
- 모든 Ai는 서로 다르고, 모든 Bi 도 서로 다르다. (두 전선의 끝점이 같은 경우가 없기 때문이다.)
- 세 전선이 한 점에서 만나지 않게 입력이 주어진다.

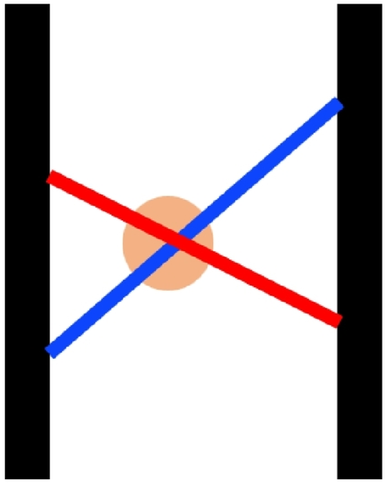
출력

- 각 테스트 케이스마다 한 줄씩 교차점의 개수를 출력하라.

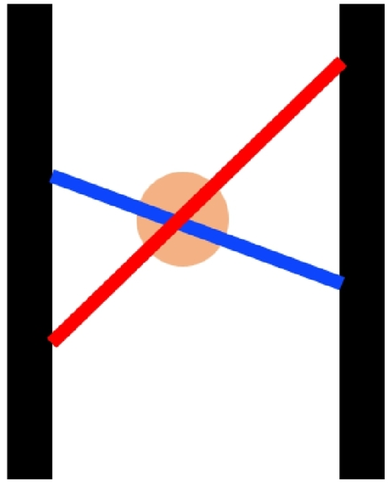
접근 방법

- 교차점이 발생하는 경우는 두 가지 이다.

기존 선
새로운 선



새로운 선의 시작점이
기존 선의 시작점보다 높으며
새로운 선의 도착점이
기존 선의 도착점보다 낮은 경우



새로운 선의 시작점이
기존 선의 시작점보다 낮으며
새로운 선의 도착점이
기존 선의 도착점보다 높은 경우

<그림5_sw문제해결 응용_문제풀이1_전봇대 접근 방법>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

1953. 탈주범 검거

문제 설명

교도소로 이송 중이던 흉악범이 탈출하는 사건이 발생하여 수색에 나섰다.

탈주범은 탈출한 지 한 시간 뒤, 맨홀 뚜껑을 통해 지하터널의 어느 한 지점으로 들어갔으며,
지하 터널 어딘가에서 은신 중인 것으로 추정된다.

<그림6_sw문제해결 응용_문제풀이1_탈주범 검거>

문제 설명

터널끼리 연결이 되어 있는 경우 이동이 가능하므로 탈주범이 있을 수 있는 위치의 개수를 계산하여야 한다.

탈주범은 시간당 1의 거리를 움직일 수 있다.

지하 터널은 총 7 종류의 터널 구조물로 구성되어 있으며
각 구조물 별 설명은 오른쪽 그림과 같다.

터널 구조물 타입	모양	기능
1		상, 하 좌, 우에 있는 터널과 연결된다.
2		상, 하에 있는 터널이 연결된다.
3		좌, 우에 있는 터널이 연결된다.
4		상, 우에 있는 터널이 연결된다.
5		하, 우에 있는 터널이 연결된다.
6		하, 좌에 있는 터널이 연결된다.
7		상, 좌에 있는 터널이 연결된다.

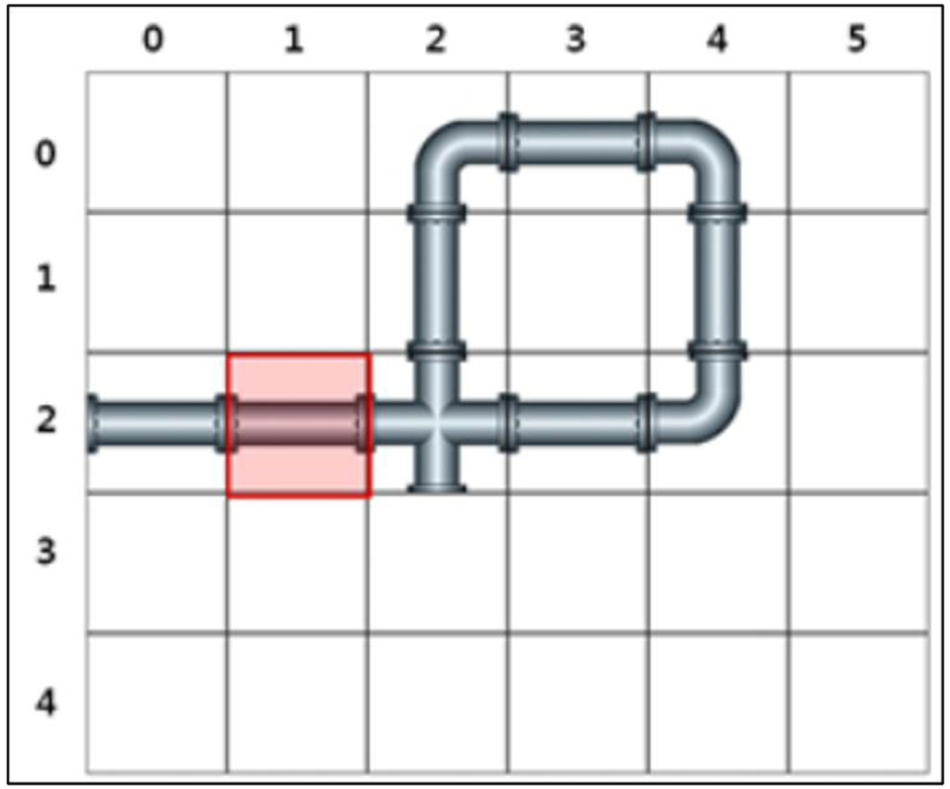
<그림7_sw문제해결 응용_문제풀이1_탈주범 검거 터널 구조물>

예제 1

그림은 지하 터널 지도의 한 예를 나타낸다.
이 경우 지도의 세로 크기는 5, 가로 크기는 6 이다.

맨홀 뚜껑의 위치가 (2, 1)로 주어질 경우,
이는 세로 위치 2, 가로 위치 1을 의미하며
붉은 색으로 표기된 구역이다.

탈주범이 **탈출 한 시간 뒤 도달할 수 있는 지점은 붉은색 한 곳**이다.

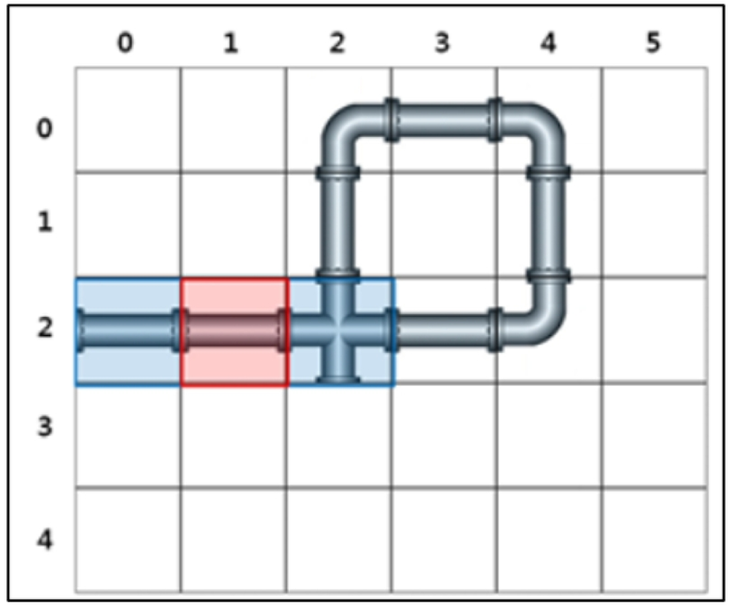


탈출 한 시간 째

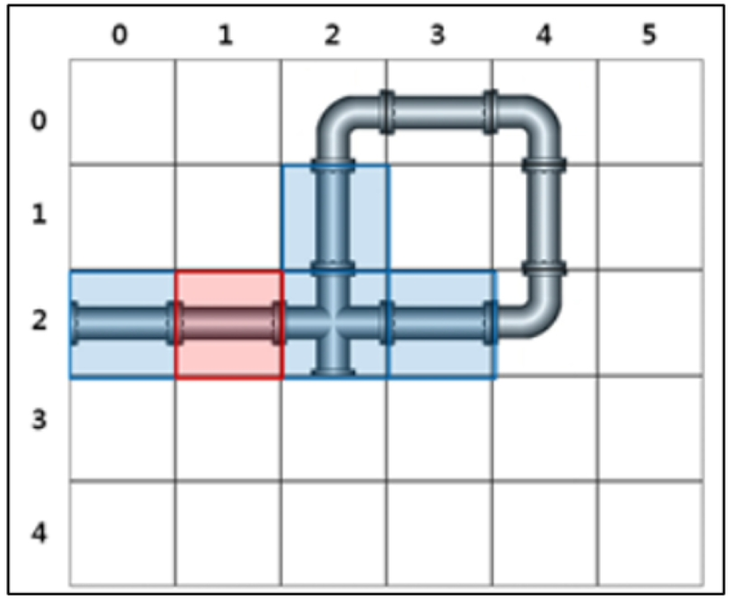
<그림8_sw문제해결 응용_문제풀이1_탈주범 검거 문제 예제 1 탈출 한 시간 째>

탈주범이 2시간 후 도달할 수 있는 지점은,

좌측 그림과 같이 맨홀 뚜껑이 위치한 붉은 색으로 표시된 지하도 와 파란색으로 표기된 지하도까지 총 3개의 장소에 있을 수 있다. 3시간 후에는 우측 그림과 같이 총 5개의 지점에 있을 수 있다.



탈출 두 시간 째

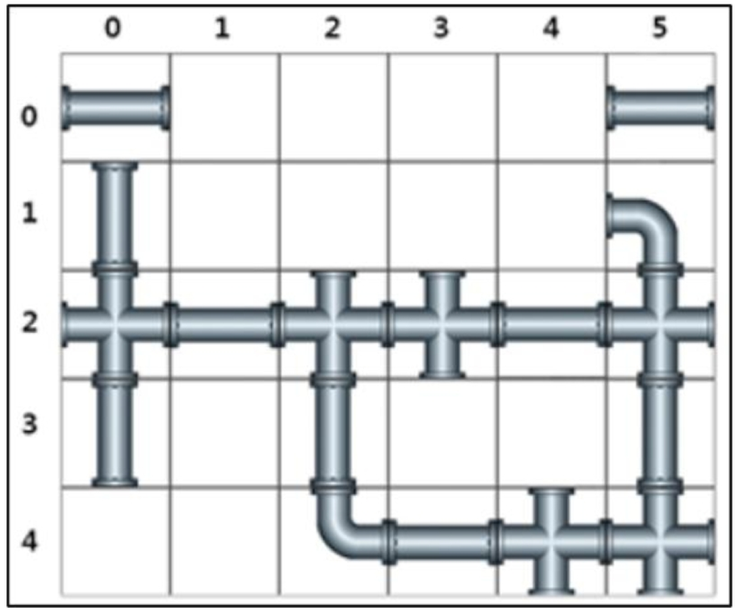


탈출 세 시간 째

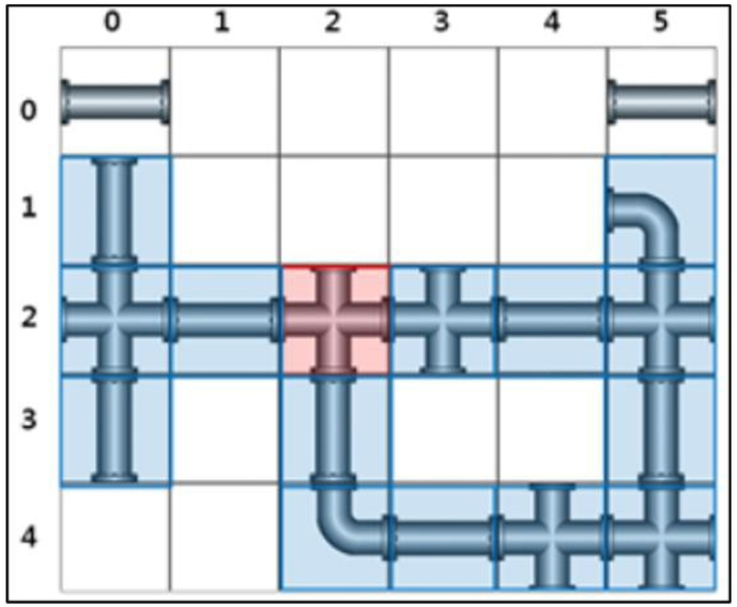
<그림9_sw문제해결 응용_문제풀이1_탈주범 검거 문제 예제 1 탈출 두 시간, 세 시간 째>

예제 2

왼쪽 그림에서 맨홀뚜껑이 위치한 지점이 (2, 2) 이고 경과한 시간이 6 으로 주어질 경우
탈주범이 있을 수 있는 장소는, 맨홀뚜껑이 위치한 지점을 포함하여 총 15 개 이다.



(2,2) 에서 시작



탈출 여섯 시간 후

<그림10_sw문제해결 응용_문제풀이1_탈주범 검거 문제 예제 2>

문제 설명

지하 터널 지도와 맨홀 뚜껑의 위치, 경과된 시간이 주어질 때
탈주범이 위치할 수 있는 장소의 개수를 계산하는 프로그램을 작성하라.

[제약 사항]

1. 50개 테스트케이스를 합쳐서 C의 경우 3초 / C++의 경우 3초 / Java의 경우 3초 / Python의 경우 15초
2. 지하 터널 지도의 세로 크기 N, 가로 크기 M은 각각 5 이상 50 이하이다. (5 ≤ N, M ≤ 50)
3. 맨홀 뚜껑의 세로 위치 R은 0 이상 N-1이하이고 가로 위치 C는 0 이상 M-1이하이다.
   (0 ≤ R ≤ N-1, 0 ≤ C ≤ M-1)
4. 탈출 후 소요된 시간 L은 1 이상 20 이하이다. (1 ≤ L ≤ 20)
5. 지하 터널 지도에는 반드시 1개 이상의 터널이 있음이 보장된다.
6. 맨홀 뚜껑은 항상 터널이 있는 위치에 존재한다.

입력

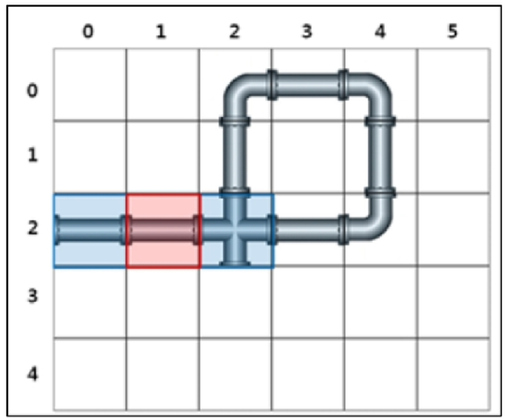
- 첫 줄에 총 테스트 케이스의 개수 T가 주어진다.
- 두 번째 줄부터 T개의 테스트 케이스가 차례대로 주어진다.
- 각 테스트 케이스의 첫 줄에는 지하 터널 지도의 세로 크기 N, 가로 크기 M, 맨홀 뚜껑이 위치한 장소의 세로 위치 R, 가로 위치 C, 그리고 탈출 후 소요된 시간 L 이 주어진다.
- 그 다음 N 줄에는 지하 터널 지도 정보가 주어지는데, 각 줄마다 M 개의 숫자가 주어진다.
- 숫자 1 ~ 7은 해당 위치의 터널 구조물 타입을 의미하며 숫자 0 은 터널이 없는 장소를 의미한다.

출력

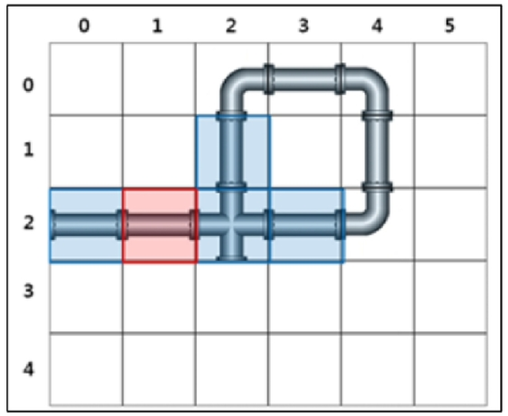
- 테스트 케이스의 개수만큼 T줄에 T개의 테스트 케이스 각각에 대한 답을 출력한다.
- 각 줄은 “#x”로 시작하고 공백을 하나 둔 다음 정답을 기록한다. (x는 1부터 시작하는 테스트 케이스의 번호이다)
- 출력해야 할 정답은 탈주범이 위치할 수 있는 장소의 개수이다.

접근 방법

- 현재 지점으로부터 갈 수 있는 지점은 연결된 파이프 1칸이다.
  - ✓ 7가지 파이프는 가능한 이동 방향이 정해져 있다. (변화량을 배열로 작성해 놓는다.)
- L 시간 전까지 현재 지점으로부터 갈 수 있는 곳을 계속 탐색한다.
  - ✓ 탐색 시 걸린 시간도 함께 기록하면서 진행한다.
- L 시간 이후 탐색된 칸 수를 카운팅한다.



탈출 두 시간 째



탈출 세 시간 째

<그림9_sw문제해결 응용_문제풀이1_탈주범 검거 문제 예제 1 탈출 두 시간, 세 시간째>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

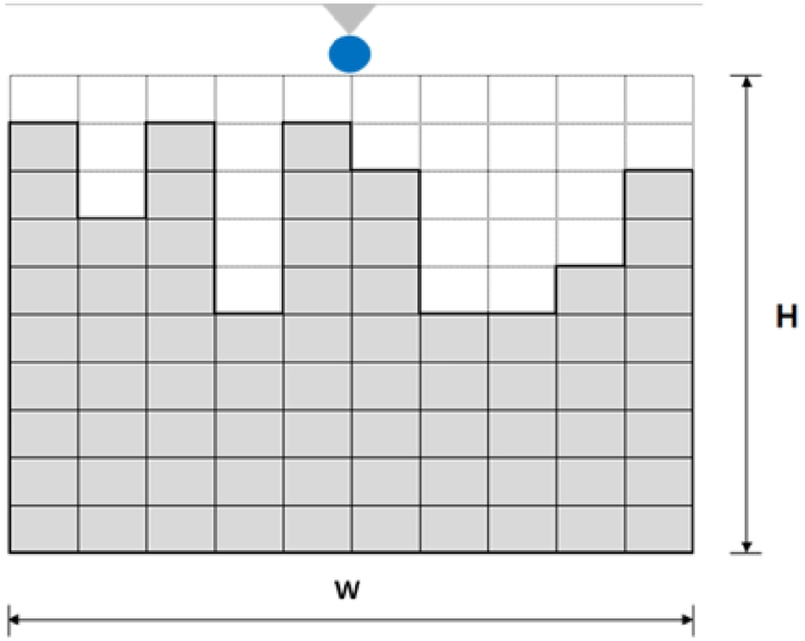
5656. 벽돌깨기

문제 설명

구슬을 쏘아 벽돌을 깨트리는 게임을 하려고 한다.

구슬은 N번만 쏠 수 있고, 벽돌들의 정보는 아래와 같이 W x H 배열로 주어진다.

(0 은 빈 공간을 의미하며, 그 외의 숫자는 벽돌을 의미한다.)



<그림11_sw문제해결 응용_문제풀이1_벽돌깨기 문제 설명 1>

문제 설명

게임의 규칙은 다음과 같다.

① 구슬은 좌, 우로만 움직일 수 있어서 항상 맨 위에 있는 벽돌만 깨트릴 수 있다.

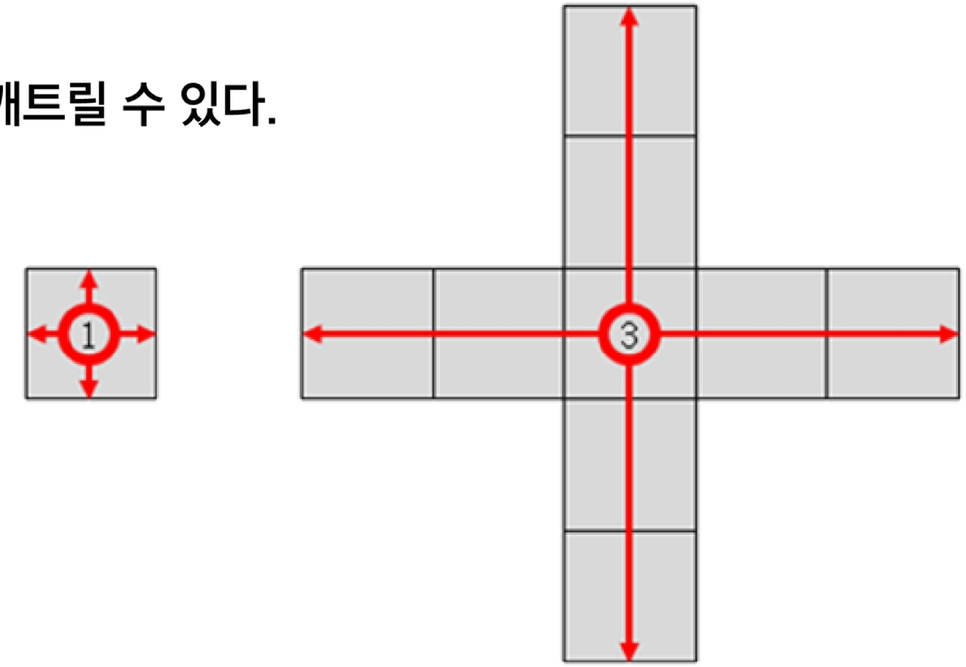
② 벽돌은 숫자 1 ~ 9 로 표현되며,

구슬이 명중한 벽돌은 상하좌우로

(벽돌에 적힌 숫자 - 1) 칸 만큼 같이 제거된다.

오른쪽은 벽돌에 적힌 숫자와, 구슬이 명중했을 시

제거되는 범위의 예이다.

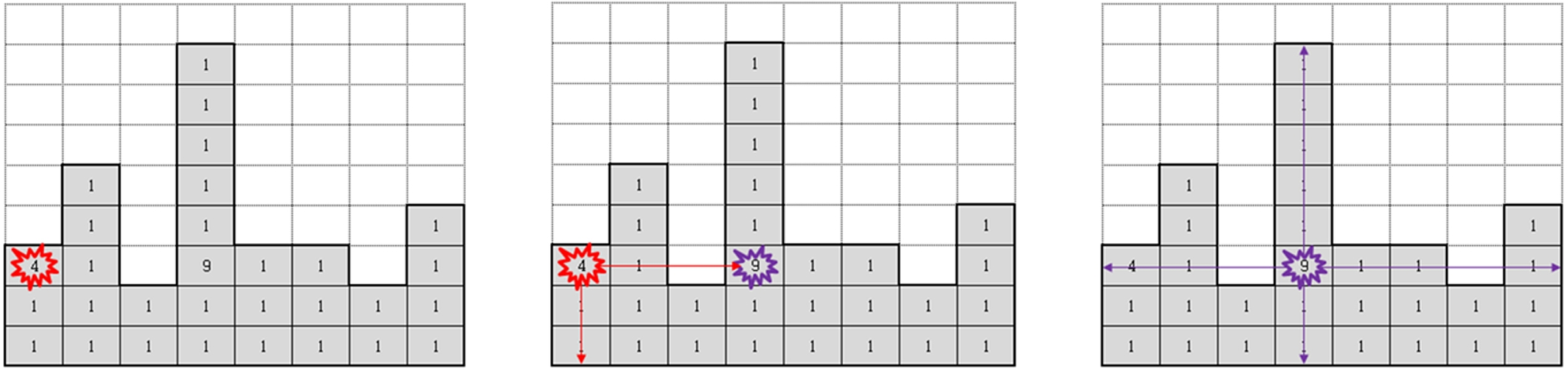


<그림12_sw문제해결 응용_문제풀이1_벽돌깨기 문제 설명 2>

문제 설명

③ 제거되는 범위 내에 있는 벽돌은 동시에 제거된다.

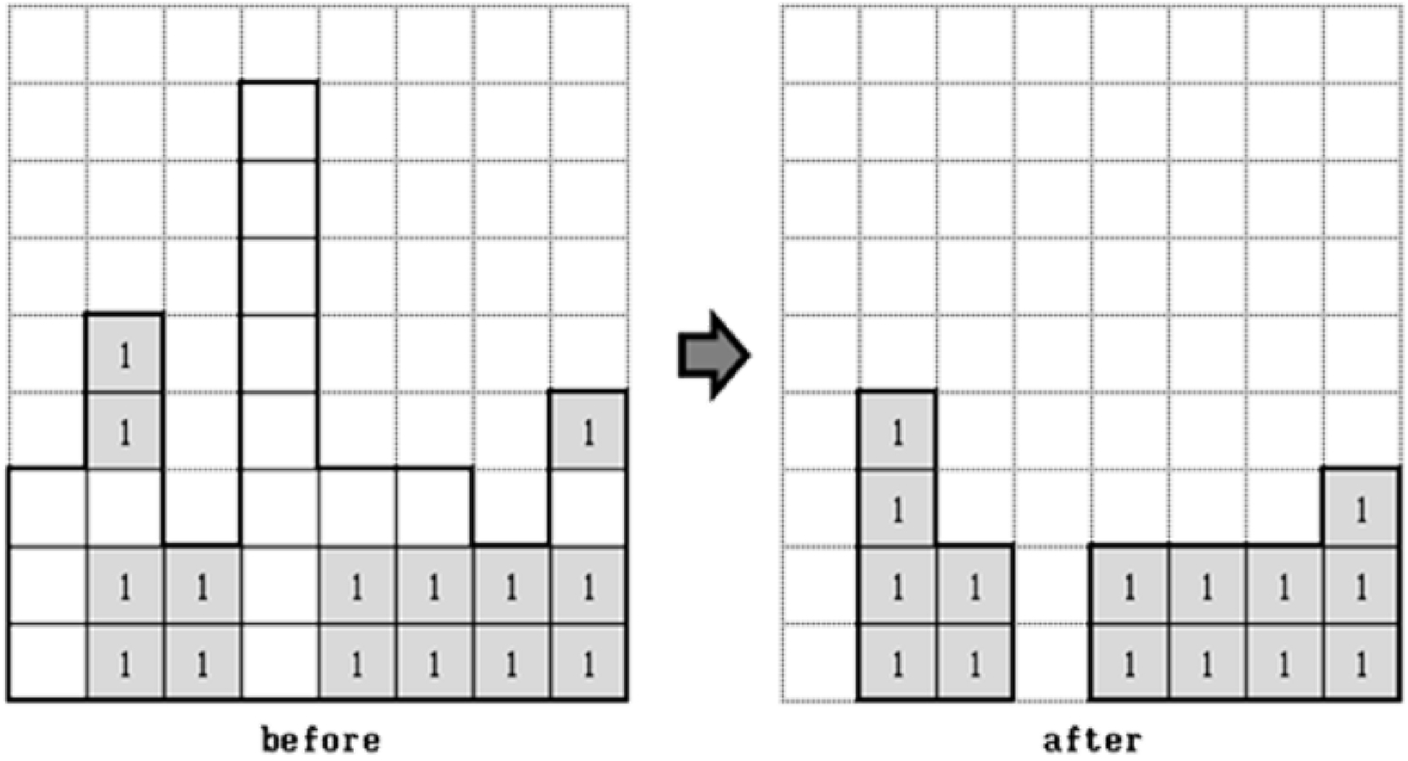
예를 들어 아래와 같이 4 번 벽돌에 구슬이 명중할 경우, 9번 벽돌은 4 번 벽돌에 반응하여, 동시에 제거된다.



<그림13_sw문제해결 응용_문제풀이1_벽돌깨기 문제 설명 3>

문제 설명

④ 빈 공간이 있을 경우 벽돌은 밑으로 떨어지게 된다.



<그림14_sw문제해결 응용_문제풀이1_벽돌깨기 문제 설명 4>

문제 설명

N 개의 벽돌을 떨어트려 최대한 많은 벽돌을 제거하려고 한다.

N, W, H, 그리고 벽돌들의 정보가 주어질 때,

▶ 남은 벽돌의 개수를 구하라!

[제약 사항]

1. 1 ≤ N ≤ 4
2. 2 ≤ W ≤ 12
3. 2 ≤ H ≤ 15

접근 방법

- 중요 규칙을 파악한다.
  - 구슬은 N번만 쏠 수 있고,
  - 구슬은 좌, 우로만 움직일 수 있어서 항상 맨 위에 있는 벽돌만 깨트릴 수 있다.

→ 모든 위치에서 시도해본다. N번 반복한다.

  - 구슬이 명중한 벽돌은 상하좌우로 (벽돌에 적힌 숫자 - 1) 칸 만큼 같이 제거된다.
  - 제거되는 범위 내에 있는 벽돌은 동시에 제거된다.

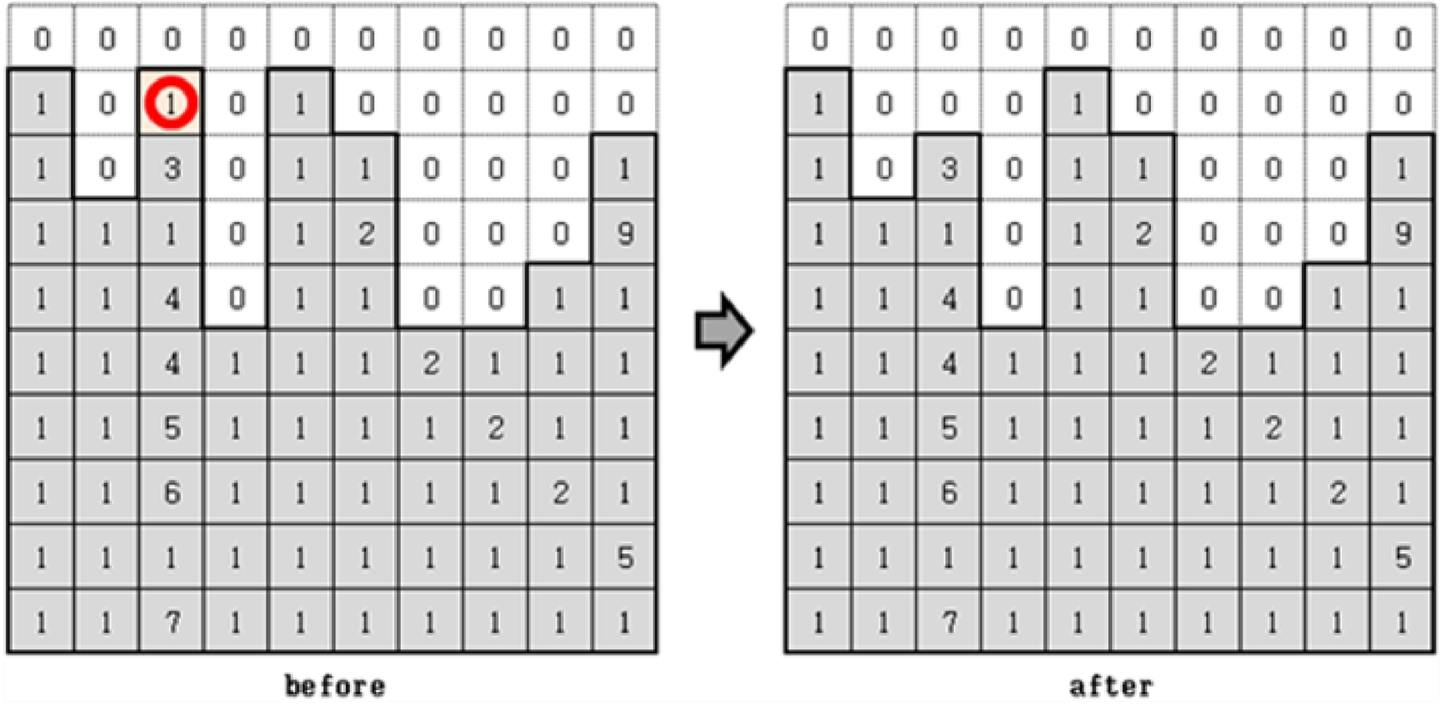
→ 추가로 제거될 벽돌의 위치와 적힌 숫자를 저장해둔다.

예제 1

- N = 3, W = 10, H = 10 이고 벽돌들의 정보가 아래와 같을 때,

  최대한 많은 벽돌을 깨트리는 방법은 아래와 같으며, 정답은 12 가 된다.

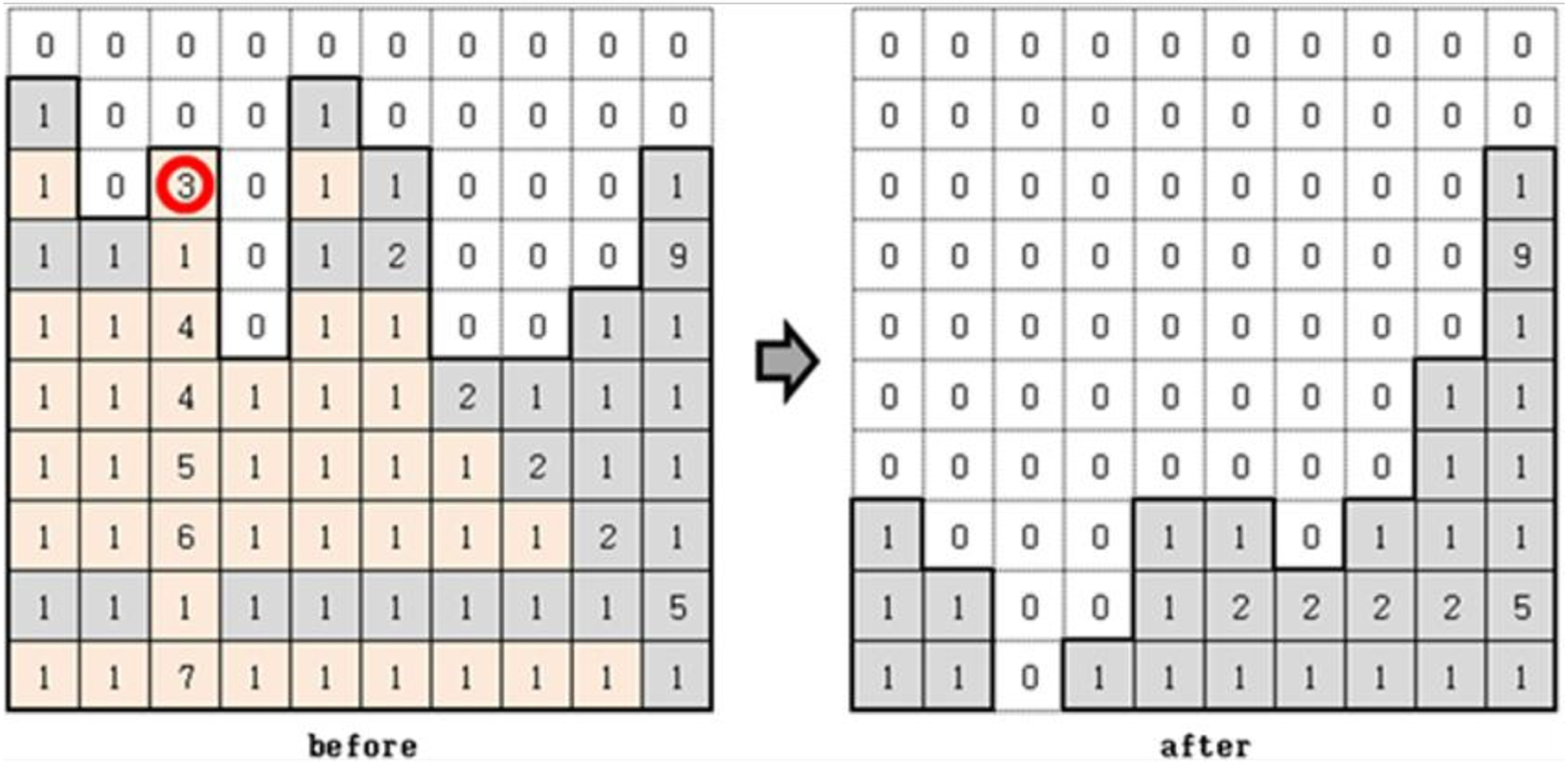
- 그림의 빨간 색 원은 구슬이 명중한 위치이며, 주황색 칸은 폭발의 범위를 의미한다.



<그림15_sw문제해결 응용_문제풀이1_벽돌깨기 예제 1 첫 번째 구슬>

예제 1

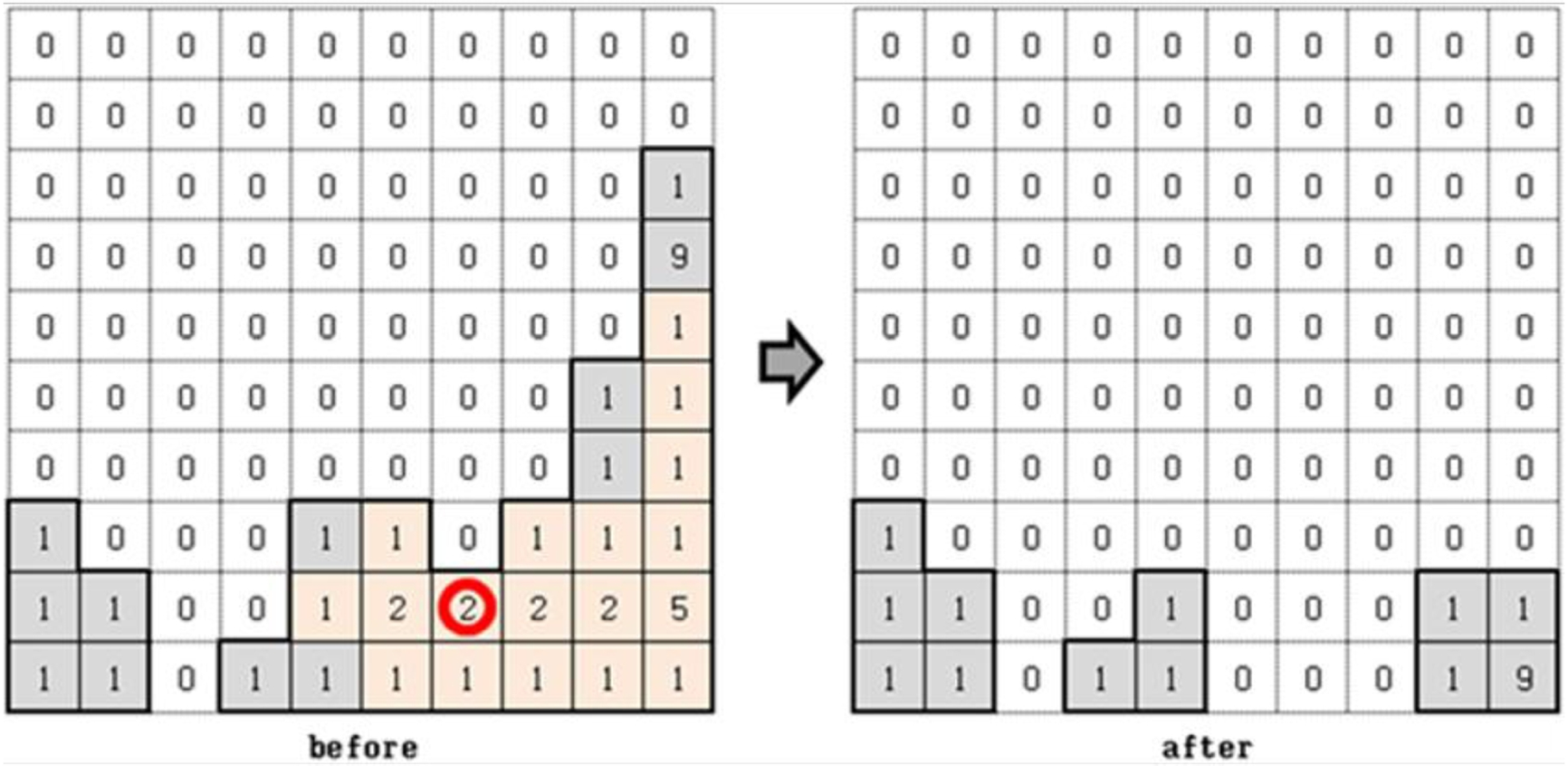
- 두 번째 구슬



<그림16_sw문제해결 응용_문제풀이1_벽돌깨기 예제 1 두 번째 구슬>

예제 1

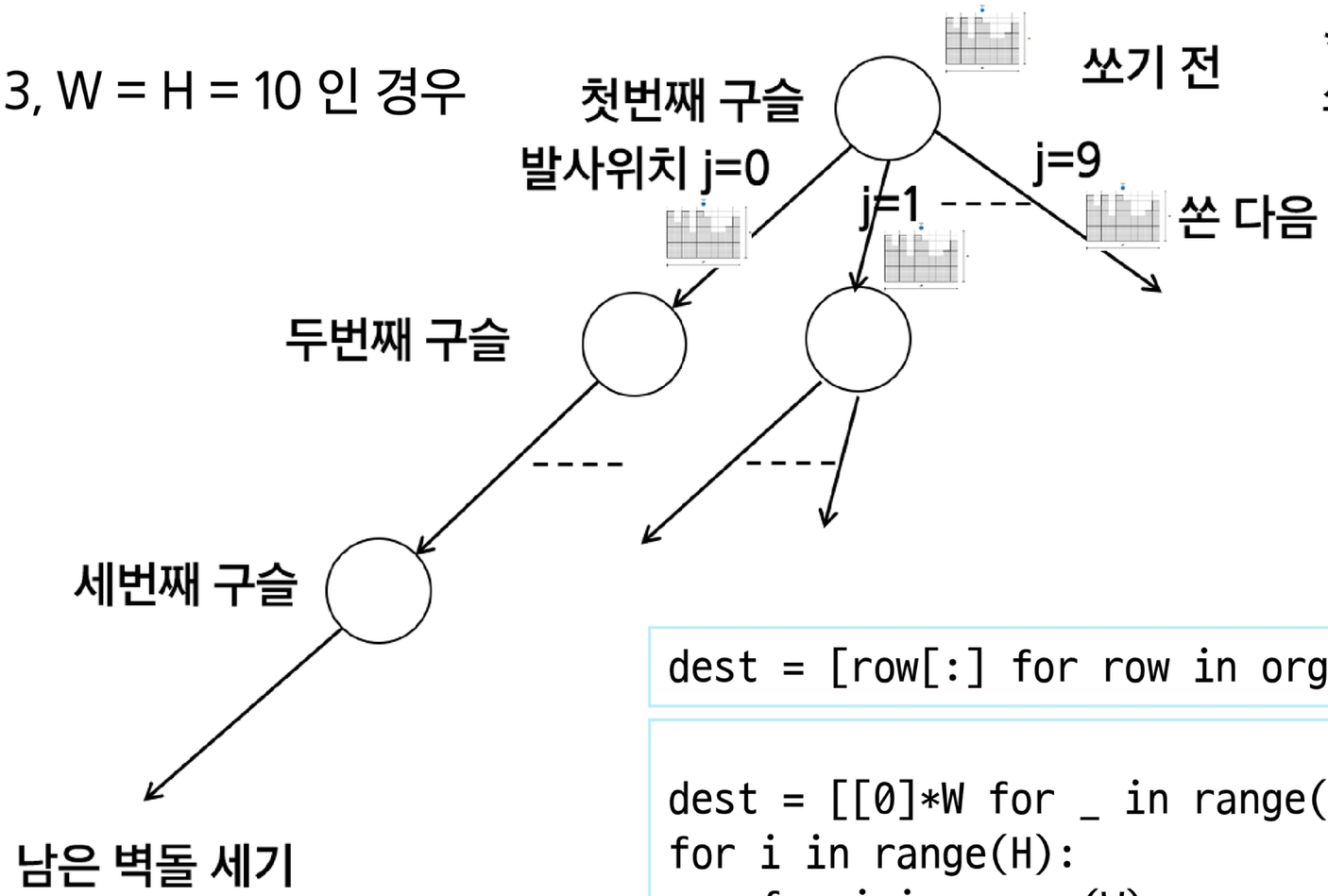
- 세 번째 구슬



<그림17_sw문제해결 응용_문제풀이1_벽돌깨기 예제 1 세 번째 구슬>

예제 1

- N = 3, W = H = 10 인 경우



*새로운 자리에 구슬을 쏠 때는
쏘기 전 기준으로 초기화 한다.

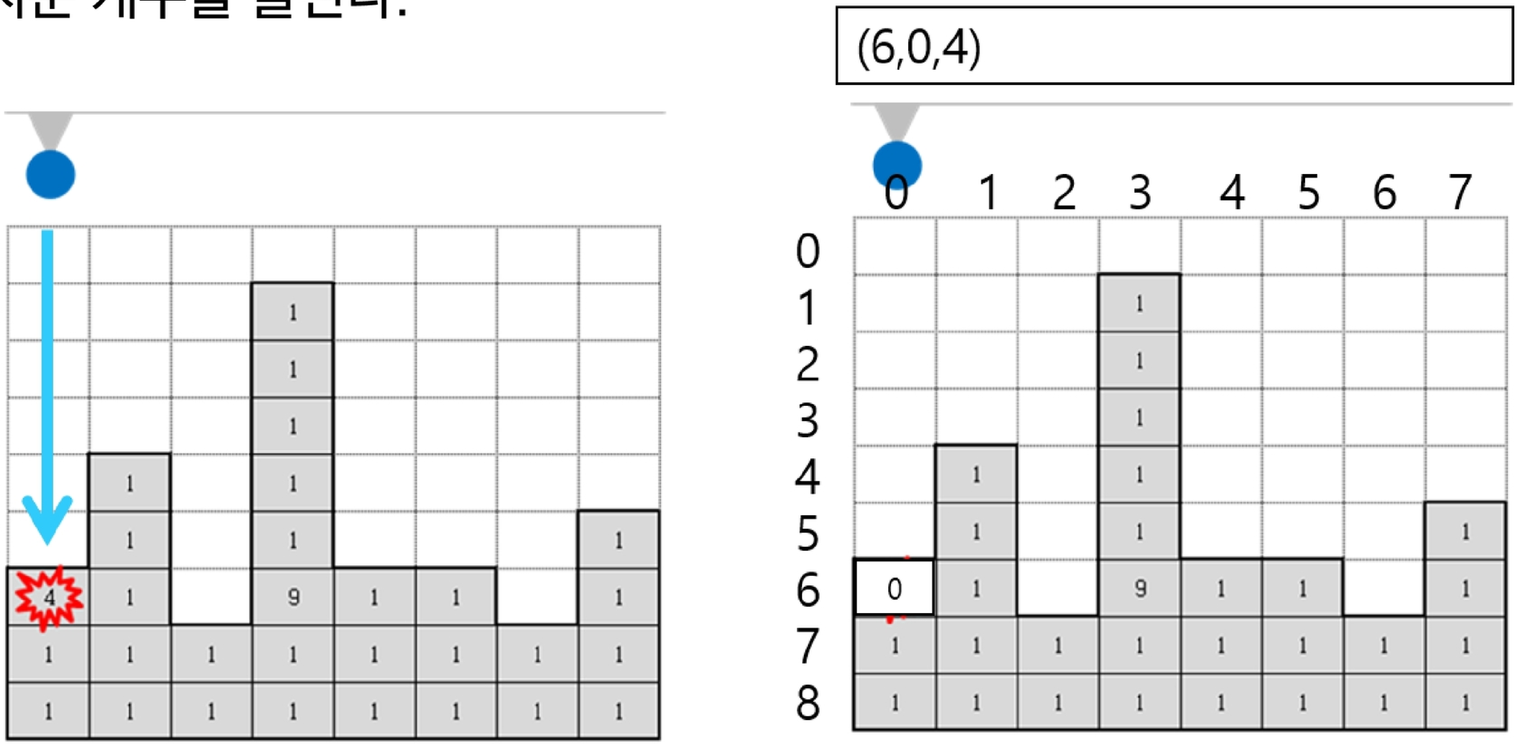
<그림18_sw문제해결 응용_문제풀이1_벽돌깨기 예제 1>

```
dest = [row[:] for row in org]        # 결과 저장용
```

```
dest = [[0]*W for _ in range(H)]  # 구슬을 발사한 결과 저장용
for i in range(H):
    for j in range(W):
        dest[i][j] = org[i][j]
```

예제 2

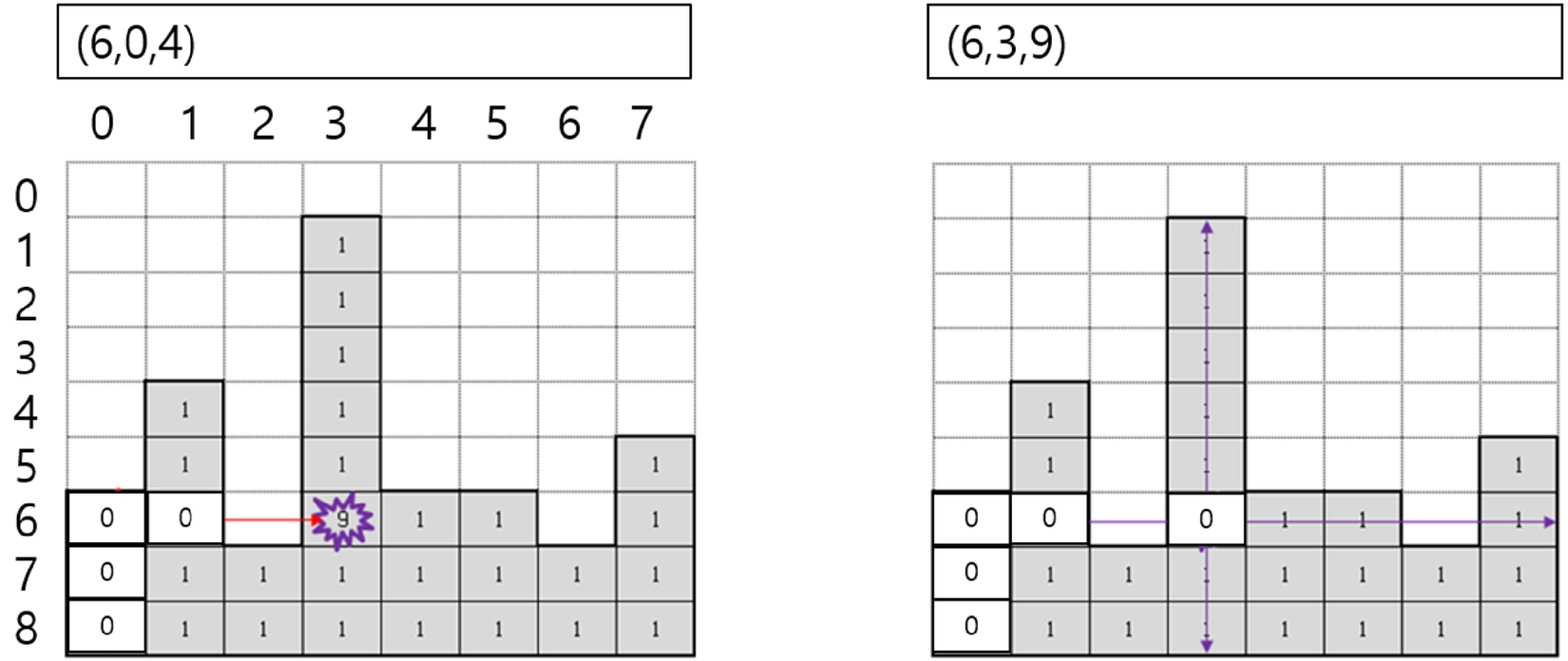
- 발사한 열에서 가장 위의 벽돌을 찾아 이동한다.
- 1보다 크면 주변에 영향을 주므로 좌표와 크기를 리스트에 저장한다.
- 벽돌을 지우고 지운 개수를 늘린다.



<그림19_sw문제해결 응용_문제풀이1_벽돌깨기 예제 2 설명 1>

예제 2

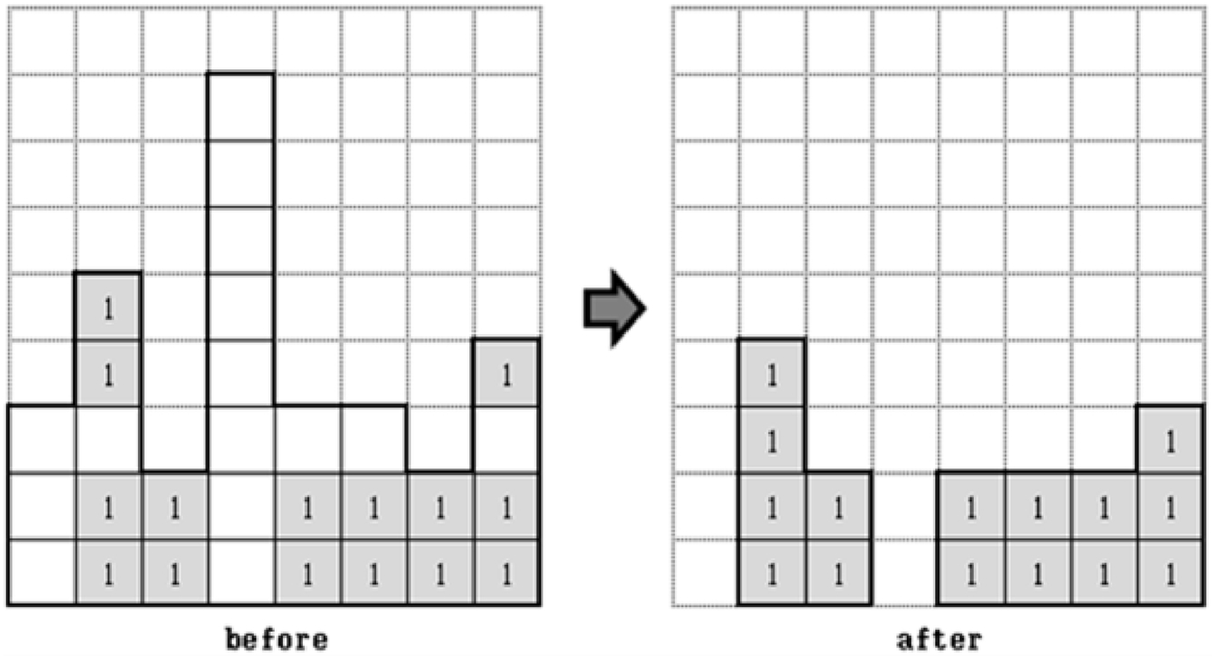
- 리스트가 비어 있지 않으면 다음을 반복한다.
- 인큐한 벽돌의 숫자만큼 네 방향으로 벽돌을 지우며 개수를 센다.
- 지워지는 벽돌이 1보다 큰 경우, 주변에 다시 영향을 주므로 정보를 저장한다.



<그림20_sw문제해결 응용_문제풀이1_벽돌깨기 예제 2 설명 2>

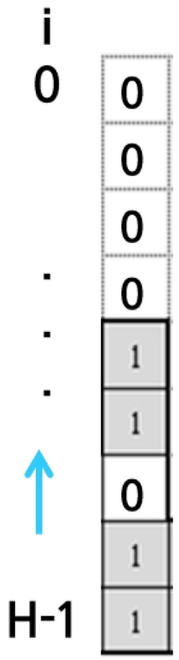
예제 2

- 지워진 자리는 위에서 내려온 벽돌로 메꾼다.



<그림21_sw문제해결 응용_문제풀이1_벽돌깨기 예제 2 설명 3>

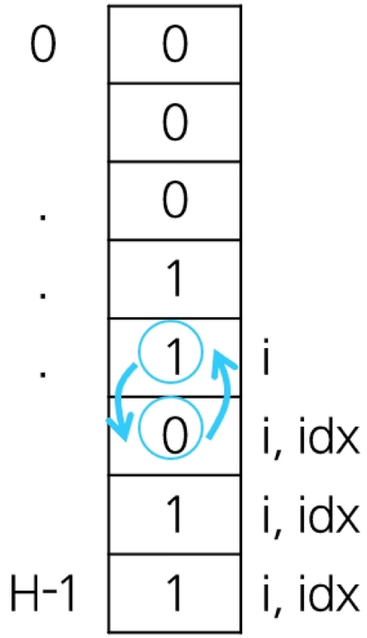
빈칸을 메우는 방법 1

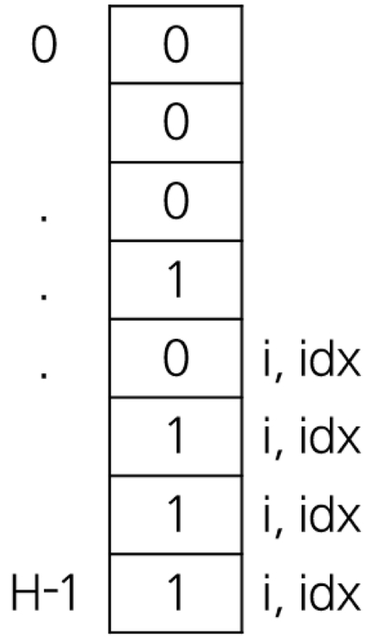



```
for j in range(W):
    for i in range(H-1, -1, -1):
        if dest[i][j] == 0:
            for l in range(i-1, -1, -1):
                if dest[l][j]:      # 벽돌이면
                    dest[i][j] = dest[l][j] # 벽돌 이동
                    dest[l][j] = 0          # 이동 후 빈자리
                    break
```

<그림22_sw문제해결 응용_문제풀이1_벽돌깨기 빈칸 메우는 방법 1>

빈칸을 메우는 방법 2





- i = idx = H-1 로 초기화
- i는 항상 감소, idx는 arr[i][j]가 0이 아닐 때만 감소
  - arr[i][j]가 0이 아니면, arr[i][j] <-> arr[idx][j] 교환 후 idx 감소

```
for j in range(W):
    idx = H - 1
    for i in range(H - 1, -1, -1):
        if dest[i][j]:
            dest[i][j], dest[idx][j] = dest[idx][j], dest[i][j]
            idx -= 1
```

<그림23_sw문제해결 응용_문제풀이1_벽돌깨기 빈칸 메우는 방법 2>

실습

큐

- 5907. 회전

시뮬레이션

- 5099. 피자 굽기

BFS

- 5102. 노드의 거리
- 5106. 미로의 거리

추가 연습

- 10966 물놀이를 가자

Test

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 질문에 답하시오. (단답형)

원래 수열의 일부 원소들을 순서대로 그대로 가져와 만든 새로운 수열

2 다음 중 BFS(너비 우선 탐색)의 특징으로 옳은 것은?

a) 한 경로 끝까지 탐색한 뒤 이동
b) 가까운 노드부터 차례로 탐색
c) 모든 경우의 수를 탐색
d) 실제 환경을 시뮬레이션

3 다음 질문에 답하시오. (단답형)

DFS 탐색 시 탐색의 진행 방식은 어떻게 이루어지나요?

4 완전 탐색(Brute Force)에 대한 설명으로 옳지 않은 것은?

a) 가능한 모든 경우를 탐색해 해답을 찾는다
b) 항상 최적해를 보장한다
c) 계산 비용이 크다
d) 실제 환경을 코드로 재현한다

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 다음 중 그래프/트리 탐색과 직접적인 관련이 없는 기법은?

a) BFS
b) DFS
c) 완전 탐색
d) 시뮬레이션

6 다음 질문에 답하시오. (단답형)

BFS를 사용할 때 가장 일반적으로 활용하는 자료구조는 무엇인가요?

7 다음 질문에 답하시오. (단답형)

DFS를 구현할 때 흔히 사용하는 자료구조는 무엇인가요?

8 다음 중 완전 탐색이 적합하지 않은 경우는?

a) 경우의 수가 적을 때
b) 최적해를 반드시 찾아야 할 때
c) 경우의 수가 매우 큰 빅데이터 상황
d) 수열의 모든 조합을 확인할 때

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 중 DFS의 활용 예로 적절한 것은?

a) 네트워크 최단 거리 탐색
b) 미로에서 출구 찾기(경로 따라가기)
c) 모든 경우의 수 조합 탐색
d) 실제 게임 환경 구현

10 다음 중 시뮬레이션 기법의 장점으로 옳은 것은?

a) 항상 최적해를 찾는다
b) 복잡한 실제 상황을 재현할 수 있다
c) 시간 복잡도가 낮다
d) 탐색 깊이에 제한이 없다

11 다음 중 BFS와 DFS의 차이로 옳지 않은 것은?

a) BFS는 큐, DFS는 스택을 사용한다
b) BFS는 최단 경로에 적합하다
c) DFS는 경우의 수 탐색에 적합하다
d) 둘 다 항상 최적해를 보장한다

12 수열 탐색 문제의 예시로 가장 적절한 것은?

a) 최장 증가 부분 수열 찾기
b) BFS로 경로 탐색하기
c) DFS로 미로 탐색하기
d) 시뮬레이션으로 게임 구현하기

확인문제

정답 및 해설

1. 부분 수열 2. b) 가까운 노드부터 차례로 탐색 3. 한 경로를 끝까지 탐색한 뒤 다른 경로로 이동한다 4. d) 실제 환경을 코드로 재현한다 5. d) 시뮬레이션 6. 큐 7. 스택(Stack) 또는 재귀(Recursion) 8. c) 경우의 수가 매우 큰 빅데이터 상황 9. b) 미로에서 출구 찾기(경로 따라가기) 10. b) 복잡한 실제 상황을 재현할 수 있다 11. d) 둘 다 항상 최적해를 보장한다 12. a) 최장 증가 부분 수열 찾기

1. 원래 수열의 원소들을 건너뛰며 선택할 수는 있지만, 선택한 원소들의 순서는 원래 순서를 유지해야 한다.

2. BFS는 가까운 노드부터 차례대로 탐색한다. DFS는 한 경로 끝까지 탐색하는 방식이다.

3. DFS(깊이 우선 탐색)는 깊게 파고든 후, 막히면 돌아와 다른 경로를 탐색한다.

4. 시뮬레이션이 실제 상황을 코드로 재현하는 방법이지, 완전 탐색은 모든 경우를 다 시도하는 방식이다.

5. 시뮬레이션은 상태 변화를 모사하는 기법으로 그래프 탐색과는 성격이 다르다.

6. BFS는 먼저 들어온 노드를 먼저 탐색하기 때문에 FIFO 구조인 큐를 사용한다.

7. DFS는 깊게 들어갔다가 다시 돌아오기 때문에 후입선출 구조의 스택이나 재귀 호출로 구현된다.

8. 경우의 수가 매우 크면 계산량이 폭발적으로 늘어나므로 완전 탐색은 비효율적이다.

9. DFS는 깊게 들어가며 탐색하므로 미로 같은 길 찾기에 자주 쓰인다.

10. 시뮬레이션은 최적해 보장보다는 실제 환경을 재현해 분석하는 것이 강점이다.

11. 목표 노드를 찾으면 탐색을 종료하는 경우 DFS는 최적해를 보장하지 않는다.

12. 수열 탐색은 주어진 수열에서 최적 부분 수열을 찾는 문제와 연관된다.

활동정리

핵심키워드

개념	설명	예시
수열 탐색	주어진 수열에서 특정 패턴이나 조건을 만족하는 원소를 찾거나 최적의 부분 수열을 찾는 기법	최장 증가 부분 수열로 전봇대 선 교차 최소화
BFS	가까운 노드부터 차례로 탐색해 나가는 그래프/트리 탐색 방법	최소 이동 횟수로 탈출 경로 찾기
DFS	한 경로를 끝까지 탐색한 뒤, 다른 경로로 이동하는 그래프/트리 탐색 방법	미로의 한 출구까지 길을 따라가며 탐색
완전 탐색	가능한 모든 경우의 수를 전부 탐색해 해답을 찾는 방법	숫자 만들기 문제에서 모든 연산 조합 시도
시뮬레이션	실제 상황의 규칙을 코드로 구현해 상태 변화를 순차적으로 재현하는 방법	벽돌깨기 게임에서 공의 충돌과 벽돌 파괴 과정 구현

<표1_sw문제해결 응용_분할정복_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **반복문과 조건**
  - 프로그램에서 동일한 작업을 여러 번 수행하거나 조건에 따라 분기 처리하는 기본 제어 구조
  - 대표적으로 for, while 반복문과 if-else 조건문을 활용해 데이터 처리 흐름 제어

- **이차원 배열 탐색과 방향 제어**
  - 행과 열로 이루어진 데이터를 상하좌우(또는 대각선)로 탐색하는 기법
  - 예: 미로 탐색에서 상·하·좌·우 이동 방향 벡터(dx, dy)를 이용한 위치 이동

활동정리

오늘 공부한 내용 요약 및 정리

- **BFS/DFS 구조 이해**
  - BFS(너비 우선 탐색): 가까운 노드부터 탐색, 최단 경로 탐색에 유리
  - DFS(깊이 우선 탐색): 한 경로를 끝까지 탐색 후 다른 경로 탐색, 백트래킹에 활용
  - BFS는 deque 모듈을 쓰면 성능 향상, DFS는 재귀 깊이 제한에 주의.

- **시뮬레이션 문제 풀이 방식**
  - 실제 상황의 규칙을 코드로 구현하여 단계별로 상태를 변화시키며 해답을 도출하는 방식

활동정리

‘스마트 시스템을 설계하는 개발자가 되어 봅시다!’

1. 사탕 수열 문제 (20551) : 데이터 흐름 중 ‘일정한 증가 패턴’을 [ ] 하는 기술→ 이상 징후 탐지, 재고 분석, 온도 변화 감지에 활용
2. 전봇대 문제 (10580) : [ ] 을 통해 선이 겹치지 않게 배선하는 방법→ 회로 설계, 통신망 구성, 배선 충돌 방지에 활용
3. 탈주범 문제 (1953) : [ ] 를 활용하여 제한된 구조에서 이동 가능한 경로 탐색→ 로봇 경로 계획, 미로 탐색, 보안 영역 침입 탐지에 활용
4. 벽돌깨기 문제 (5656) : BFS/DFS를 활용하여 하나의 행동이 주변에 미치는 영향을 [ ] → 재난 예측, 구조물 붕괴 예측, 게임 물리 엔진 설계에 활용

활동정리

‘스마트 시스템을 설계하는 개발자가 되어 봅시다!’

1. 사탕 수열 문제 (20551) : 데이터 흐름 중 ‘일정한 증가 패턴’을 탐색하는 기술→ 이상 징후 탐지, 재고 분석, 온도 변화 감지에 활용
2. 전봇대 문제 (10580) : 수열 탐색을 통해 선이 겹치지 않게 배선하는 방법→ 회로 설계, 통신망 구성, 배선 충돌 방지에 활용
3. 탈주범 문제 (1953) : BFS를 활용하여 제한된 구조에서 이동 가능한 경로 탐색→ 로봇 경로 계획, 미로 탐색, 보안 영역 침입 탐지에 활용
4. 벽돌깨기 문제 (5656) : BFS/DFS를 활용하여 하나의 행동이 주변에 미치는 영향을 시뮬레이션→ 재난 예측, 구조물 붕괴 예측, 게임 물리 엔진 설계에 활용

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다